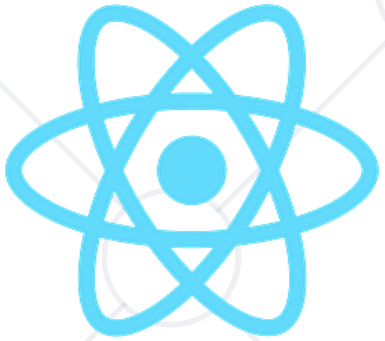


Navigation

Building the Backbone of Mobile User Flow



React Native

SoftUni Team
Technical Trainers



SoftUni



Software University

<https://softuni.bg>

Table of Contents

1. Navigation Fundamentals
2. Core Concepts
3. Stack Navigation
4. Moving Between Screens
5. Passing Data Between Screens
6. More Navigators





Navigation Fundamentals

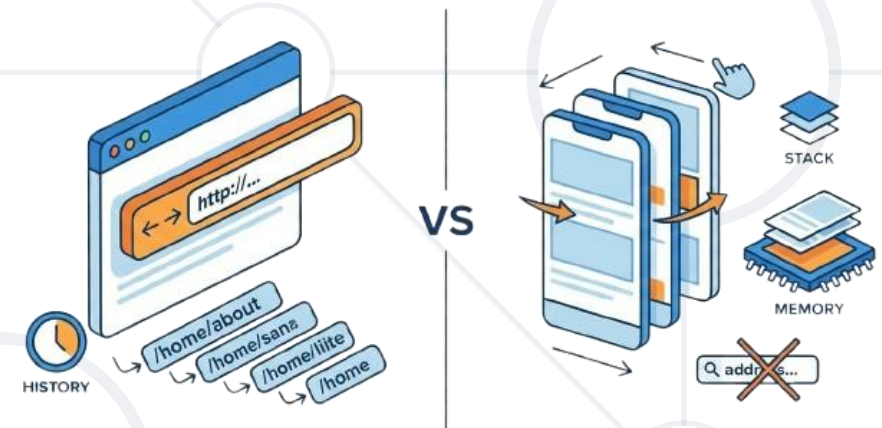
What is Navigation?

- 
- Navigation defines how users move between screens in a mobile application
 - Screen-based, **not** URL-based
 - Navigation functions: **push**, **forward**, **backward**
 - Navigation defines structure, **UX** and **flows**
 - Navigation is **routing**, **state** and **history**
 - Mobile navigation **≠** web routing

Navigation is the backbone of user flow.

Web Routing vs Mobile Navigation

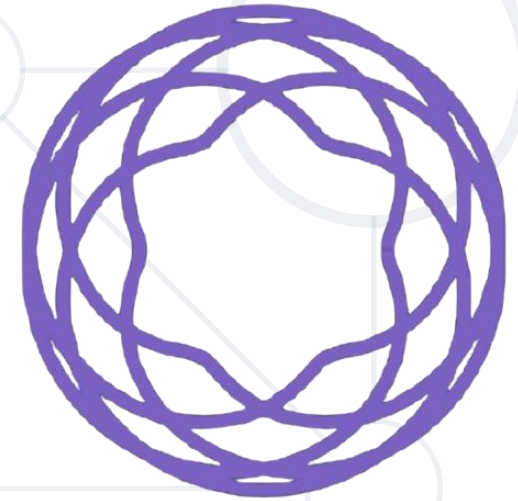
- Mobile navigation is **stateful** and **stack-based**, not URL-based
 - **Web** uses URLs and browser history
 - **Mobile** uses **navigation stacks**
 - **No address bar** in mobile apps
 - Screens stay **mounted** in memory
 - **Gestures** are part of navigation



Think in screens and stacks, not URLs.

Why React Navigation?

- React Navigation is the de-facto navigation solution for React Native apps.
 - Pure **JavaScript** implementation
 - Works with **Expo** and **CLI**
 - Supports **stacks, tabs, drawers**
 - **Gesture-based** navigation
 - **Large** ecosystem and community



React Navigation is the industry standard.

- **React Router Native** brings React Router concepts to React Native apps
 - Based on **React Router** (web-first mental model)
 - Uses **routes and paths** instead of stacks
 - Familiar API for web developers
 - **Limited** mobile-native patterns
 - Much **smaller** ecosystem than **React Navigation**



React Router works for React Native, but React Navigation is built for mobile.

- **Expo Router** is a file-based routing system built on top of React Navigation
 - File-system–based routing (**app/ folder**)
 - Inspired by **Next.js routing**
 - Built specifically for **Expo**
 - Automatically configures navigators
 - Uses **React Navigation** under the hood



Expo Router

*Expo Router simplifies navigation setup,
but React Navigation concepts still matter.*



Core Concepts

- **NavigationContainer** manages the navigation tree and state
 - Must **wrap** your entire app
 - **Stores** navigation state
 - Handles **deep** linking
 - Enables screen transitions
 - **Only one** container per app

```
<NavigationContainer>  
<AppNavigator />  
</NavigationContainer>
```

No container, no navigation.

- A **navigator** is a component that manages how screens are organized and transitioned
 - Defines **navigation behavior**
 - Controls **screen transitions**
 - Manages **navigation state**
 - Decides **history and gestures**
 - Screens must live inside a navigator

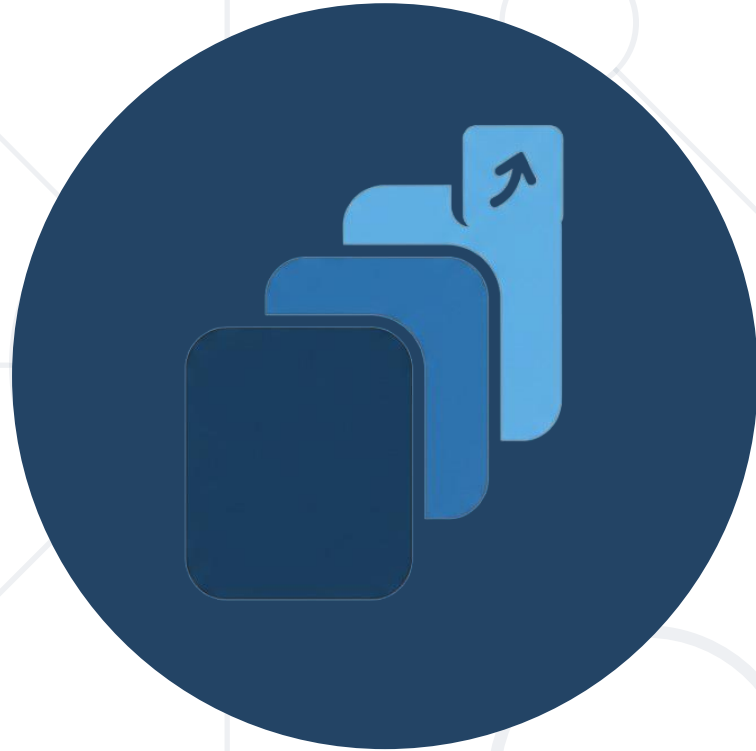
Navigators define how screens move, not just which screens exist.

- A **screen** is a React component registered in a navigator
 - Each screen **maps** to a component
 - Screens receive **navigation** and **route props**
 - Screen **name** is the navigation key
 - Screens can be **reused**
 - Screens define **app flow**

Screens are the building blocks of navigation.

- React Navigation can be **configured** either **statically** or **dynamically**
 - **Static configuration**
 - Defined when the navigator is created
 - Same behavior for all renders
 - Predictable and easy to reason about
 - **Dynamic configuration**
 - Depends on props, state, or route params
 - Can change per screen or per render
 - Useful for conditional UI or behavior

Use static config by default, dynamic config when behavior depends on state.



Stack Navigation

What is Stack Navigation?

- **Stack navigation** mimics a call stack:
last in, first out
 - Screens stack on **top of each other**
 - Supports **back** navigation
 - **Default pattern** for mobile apps
 - Header included by **default**
 - Gesture-based transitions

Stack navigation models natural mobile flow.



Creating a Stack Navigator

- Stacks are created using **createNativeStackNavigator**
 - Fast native performance
 - Automatic headers
 - Platform-appropriate transitions
 - Simple API
 - Ideal for most screens

```
const Stack = createNativeStackNavigator();  
<Stack.Navigator>  
  <Stack.Screen name="Home" component={HomeScreen} />  
</Stack.Navigator>
```

Start with a stack, always.

- Options control headers, animations, and behavior
 - Control **screen presentation**, not layout
 - Configure **headers and titles**
 - Define **transition style**
 - Decide **how the screen enters and exits**

```
<Stack.Screen  
  name="Details"  
  component={DetailsScreen}  
  options={{ headerShown: false }}  
/>
```

Options shape the navigation experience.

- Customizing the **Stack Header**
 - Header is provided by the **stack navigator**
 - Can be shown or hidden per screen
 - Title can be **static** or **dynamic**
 - Header buttons enable actions
 - Header reflects navigation state

```
<Stack.Screen  
  name="Profile"  
  component={ProfileScreen}  
  options={{  
    title: 'My Profile',  
    headerTintColor: #fff,  
  }}  
>
```

The header is part of navigation, not screen layout.

- Navigation options can be defined at multiple levels, and React Navigation resolves them by **priority**
 - Options can be set on the **Navigator**
 - Options can be set on a **Group**
 - Options can be set on a **Screen**
 - More specific options **override** broader ones
 - Screen options have the **highest priority**

The closer the option is to the screen, the higher its priority.



Moving Between Screens

- Screens can be connected **declaratively** using navigation-aware components.
 - Navigation triggered by **UI components**
 - No imperative logic **required**
 - Similar to **links** on the web
 - Clear intent in **JSX**

```
<Link screen="Details"> Go to Details </Link>  
<Button screen="Details"> Go to Details </Button>
```

Use links and buttons when navigation is part of the UI structure.

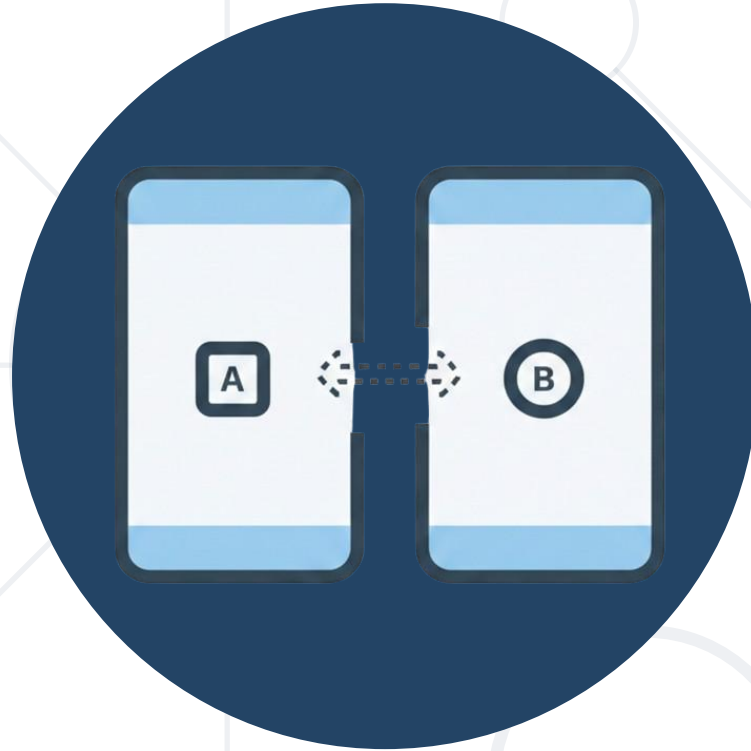
- Screens move between each other using navigation actions triggered by user interaction or logic.
 - Navigation is triggered **imperatively**
 - Most commonly happens on **user actions**
 - Uses the **navigation** object
 - Can move forward, backward, or replace screens
 - Prop vs Hook

```
<Button title="Go to Details"  
  onPress={() => navigation.navigate('Details')}  
>
```

Navigation is an action, not a render.

- Returning to Previous Screens
 - Uses the navigation **stack history**
 - Can be triggered by **UI or gestures**
 - Works **differently** across navigators
 - **Removes** current screen from the stack

```
<Button title="Go Back"  
  onPress={() => navigation.goBack()}  
/>
```



Passing Data Between Screens

- **Route params** allow passing data between screens
 - Sent via **navigate**
 - Send via **prop**
 - Serializable values only
 - Common for **IDs** and **flags**

```
navigation.navigate('Details', { id:42 });
```

Navigation is also data flow.

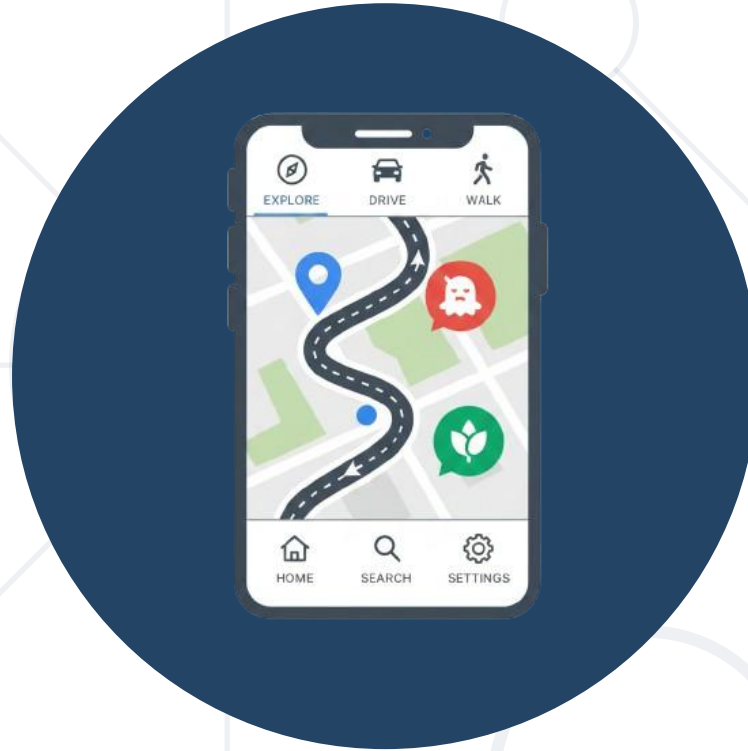
- Each screen receives **params** through the route prop
 - **route.params** contains data
 - Params **persist** in navigation state
 - Can be optional
 - Useful for **dynamic screens**
 - **useRoute** hook

```
const { id } = route.params;
```

Screens react to navigation context.

- Params can be **updated** after navigation
 - **setParams()** updates route data
 - Triggers **re-render**
 - Useful for screen state sync

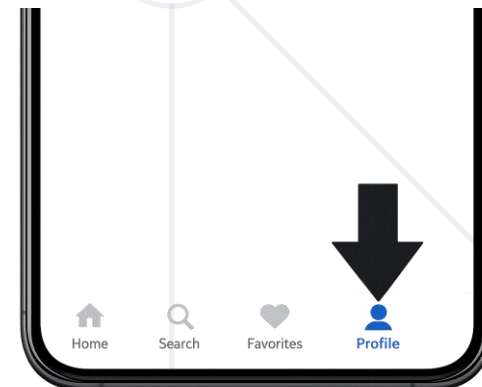
Params are screen-specific state.



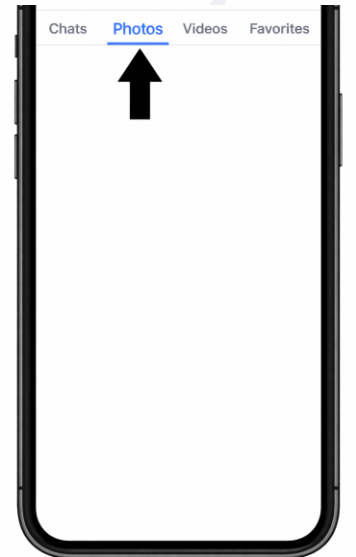
More Navigators

- Tabs allow switching between **top-level** screens
 - **Persistent** navigation bar
 - No stacking by **default**
 - Ideal for main sections
 - **Common** in consumer apps
 - Usually bottom tabs

Tabs are for parallel navigation.

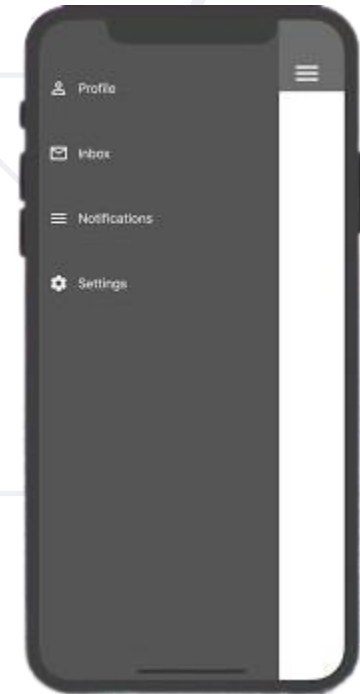


- Material Top Tabs display tabs at the top of the screen, following **Material Design** patterns
 - Tabs appear **at the top**
 - Swipe gestures between screens
 - Built on **react-native-pager-view**
 - Common on Android
 - Ideal for **content sections** within a screen



Top tabs are great for peer content with swipe navigation.

- Side Menu Navigation
 - Slides from screen edge
 - **Common** for settings and admin
 - Gesture-driven
 - Can **wrap** other navigators
 - **Less** common in modern apps



Drawers are for secondary access.

- Navigators can be **nested** inside each other
 - Stack inside tabs
 - Tabs inside stack
 - Drawer wrapping everything
 - **Enables** complex flows
 - Very **common** pattern

Navigation is composable, like components.

- Mobile navigation is stateful and stack-based, serving as the back bone of user flow
- React Navigation is the industry standard, utilizing a central **NavigationContainer** to manage screen transitions and history
- Core patterns include **Stack**, **Tab**, and **Drawer** navigators, which can be nested to build complex, composable architectures
- Data flow and logic are handled via route parameters, dynamic headers, and specific configuration options to ensure seamless user journeys



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

